

JDBC_Sink2



Warning

En este apartado se supone una base de datos *kafka* en un contenedor accesible para el *ksqldb-server*

Prerequisitos

- Los datos ya están cargados en el cluster *Kafka* (continuación del tutorial [JDBCSink1](#))
- Las tablas *albsong1* y *albsong2* ya han sido creadas en la base de datos *kafka* (ver [ddlKafka](#))

Claúsula *AS_VALUE*

```
-- Mensajes Kafka: pares clave/valor
-- Etiquetas k1 y k2 para evitar el duplicado de nombres
CREATE TABLE `AlbSong1`
WITH (KAFKA_TOPIC='albsong1', KEY_FORMAT='AVRO', VALUE_FORMAT='AVRO')
AS SELECT
    artist as k1,
    album as k2,
    AS_VALUE(artist) AS artist,
    AS_VALUE(album) AS album,
    count(*) as N
FROM `raw_songs`
WHERE title IS NOT NULL
GROUP BY artist, album;

-- Resultados
print 'albsong1' from beginning limit 2;
select * from `AlbSong1`;
select artist, album, n from `AlbSong1`;
```

Resultado del *print*

```
Key format: AVRO or HOPPING(KAFKA_STRING) or TUMBLING(KAFKA_STRING) or
KAFKA_STRING
Value format: AVRO or KAFKA_STRING
-----
-----
rowtime: 2023/03/21 10:51:48.082 Z,
key: {"K1": "Soundgarden", "K2": "Superunknown"},
```

```
value: {"ARTIST": "Soundgarden", "ALBUM": "Superunknown", "N": 1}, partition: 0
rowtime: 2023/03/21 10:51:48.082 Z,
key: {"K1": "Nirvana", "K2": "Nevermind"},
value: {"ARTIST": "Nirvana", "ALBUM": "Nevermind", "N": 3}, partition: 0
```

Definir conector *JDBCSink*

```
CREATE SINK CONNECTOR `sink-jdbc-albsong1` WITH(
  "connector.class" = 'io.confluent.connect.jdbc.JdbcSinkConnector',
  "connection.url" = 'jdbc:mysql://mysql:3306/kafka',
  "topics" = 'albsong1',
  "table.name.format" = 'albsong1',
  "key.converter" = 'io.confluent.connect.avro.AvroConverter',
  "key.converter.schema.registry.url" = 'http://schema-registry:8081',
  "key.converter.schemas.enable" = 'true',
  "value.converter" = 'io.confluent.connect.avro.AvroConverter',
  "value.converter.schema.registry.url" = 'http://schema-registry:8081',
  "value.converter.schemas.enable" = 'true',
  "connection.user" = 'kafka',
  "connection.password" = 'kafka',
  "auto.create" = 'false',
  "pk.mode" = 'record_value',
  "pk.fields" = 'ARTIST,ALBUM',
  "insert.mode" = 'upsert',
  "delete.enabled" = 'false',
  "tasks.max" = '1');
```

Datos Estructurados

La consulta a continuación muestra el uso de la palabra reservada **STRUCT** para componer una columna **k** compuesta por los campos *artist* y *album*. El resultado de la consulta muestra el nº de canciones en los discos de cada artista

```
SELECT struct(artist := artist, album := album) as k,
       count(*) as N
FROM `raw_songs`
WHERE title IS NOT NULL AND album IS NOT NULL
GROUP BY struct(artist := artist, album := album)
emit changes;
```

A continuación, la misma consulta anterior es utilizada para componer una tabla

```
CREATE TABLE `AlbSong2`
WITH (KAFKA_TOPIC='albsong2', KEY_FORMAT='AVRO', VALUE_FORMAT='AVRO')
AS SELECT
  struct(artist := artist, album := album) as k,
  count(*) as N
FROM `raw_songs`
```

```
WHERE title IS NOT NULL AND album IS NOT NULL
GROUP BY struct(artist := artist, album := album);
```

La estructura de los mensajes en el t3pico que corresponde a la tabla es conforme a lo siguiente:

```
print 'albsong2' from beginning limit 2;
-----
-----
Key format: AVRO or HOPPING(KAFKA_STRING) or TUMBLING(KAFKA_STRING) or
KAFKA_STRING
Value format: AVRO or KAFKA_STRING
rowtime: 2023/02/24 13:01:58.648 Z, key: {"K1": "Soundgarden", "K2":
"Superunknown"}, value: {"N": 1}
rowtime: 2023/02/24 13:01:58.648 Z, key: {"K1": "Nirvana", "K2": "Nevermind"},
value: {"N": 3}
```

Se observa que el campo clave del mensaje almacena los valores del *struct* con las etiquetas *K1* y *K2* en tanto que en el campo valor 3nicamente aparece el agregado. La organizaci3n interna de los mensajes es exactamente la misma si la consulta se expresa sin *struct*:

```
CREATE TABLE `AlbSong3`
WITH (KAFKA_TOPIC='albsong3', KEY_FORMAT='AVRO', VALUE_FORMAT='AVRO')
AS SELECT
    artist,
    album,
    count(*) as N
FROM `raw_songs`
WHERE title IS NOT NULL AND album IS NOT NULL
GROUP BY artist, album;
```

Sin embargo, en el primer caso la tabla tiene 3nicamente dos columnas mientras que en el 3ltimo tiene tres:

```
ksql> select * from `AlbSong2`;
+-----+-----+
|          K          | N |
+-----+-----+
| {ARTIST=Nirvana, ALBUM=Nevermind} | 3 |
| {ARTIST=Pearl Jam, ALBUM=Ten}      | 4 |
| {ARTIST=Soundgarden, ALBUM=Superunknown} | 1 |
| {ARTIST=Tool, ALBUM=Aenima}        | 1 |
| {ARTIST=Tool, ALBUM=Lateralus}     | 1 |
+-----+-----+

ksql> select * from `AlbSong3`;
+-----+-----+-----+
| ARTIST | ALBUM | N |
+-----+-----+-----+
```

Nirvana	Nevermind	3	
Pearl Jam	Ten	4	
Soundgarden	Superunknown	1	
Tool	Aenima	1	
Tool	Lateralus	1	

Conector *JDBCSink*



OJO

- ***pk.mode***: la del mensaje
- ***pk.fields***: no hay que especificar nada, coincide con la de la tabla
- ***insert.mode***: *upsert* (*update* + *insert*)
- ***white list***: el agregado

```
-- Crear conector desde ksql
-- Suponiendo creada la tabla en la base de datos

CREATE SINK CONNECTOR `sink-jdbc-albsong2` WITH(
  "connector.class" = 'io.confluent.connect.jdbc.JdbcSinkConnector',
  "connection.url" = 'jdbc:mysql://mysql:3306/kafka',
  "topics" = 'albsong2',
  "table.name.format" = 'albsong2',
  "key.converter" = 'io.confluent.connect.avro.AvroConverter',
  "key.converter.schema.registry.url" = 'http://schema-registry:8081',
  "key.converter.schemas.enable" = 'true',
  "value.converter" = 'io.confluent.connect.avro.AvroConverter',
  "value.converter.schema.registry.url" = 'http://schema-registry:8081',
  "value.converter.schemas.enable" = 'true',
  "connection.user" = 'kafka',
  "connection.password" = 'kafka',
  "auto.create" = 'false',
  "pk.mode" = 'record_key',
  "pk.fields" = '',
  "fields.withelists" = 'N',
  "insert.mode" = 'upsert',
  "delete.enabled" = 'true',
  "tasks.max" = '1');
```

Para entender la relevancia del valor *upsert* en *insert.mode*, además de consultar la documentación relativa a la [configuración del conector](#), se recomienda añadir al directorio donde lee el conector *connect-file-pulse-csv* definido en [Musiteca](#) un archivo con el contenido siguiente:

```
title;album;duration;release;artist;type
Spoonman;Superunknown;04:06;1994;Soundgarden;Rock
```

El resultado es que el conector captura la línea del fichero y los datos fluyen internamente como mensajes en los tópicos correspondientes hasta que *albsong2* (el tópico que se envía a la base de datos) contiene lo siguiente:

```
ksql> print 'albsong2' from beginning;
-----
Key format: AVRO or HOPPING(KAFKA_STRING) or TUMBLING(KAFKA_STRING) or
KAFKA_STRING
Value format: AVRO or KAFKA_STRING
rowtime: 2025/02/19 13:51:08.572 Z, key: {"ARTIST": "Soundgarden", "ALBUM":
"Superunknown"}, value: {"N": 1}
rowtime: 2025/02/19 13:51:08.572 Z, key: {"ARTIST": "Nirvana", "ALBUM":
"Nevermind"}, value: {"N": 3}
rowtime: 2025/02/19 13:51:08.572 Z, key: {"ARTIST": "Pearl Jam", "ALBUM":
"Ten"}, value: {"N": 4}
rowtime: 2025/02/19 13:51:08.572 Z, key: {"ARTIST": "Tool", "ALBUM":
"Aenima"}, value: {"N": 1}
rowtime: 2025/02/19 13:51:08.572 Z, key: {"ARTIST": "Tool", "ALBUM":
"Lateralus"}, value: {"N": 1}
rowtime: 2025/02/19 15:09:08.696 Z, key: {"ARTIST": "Soundgarden", "ALBUM":
"Superunknown"}, value: {"N": 2}
```

Es decir, los valores del campo clave en los mensajes primero y último coinciden de modo que un *insert* en la base de datos no funcionaría correctamente

Claúsula *PARTITION BY*

Las claves estructuradas son especialmente interesantes cuando se utilizan para controlar el particionado de los datos lo cual es determinante en el rendimiento global del sistema. A continuación se muestra la definición de un *stream* análogo a los precedentes pero añadiendo una clave de particionado

```
CREATE STREAM part_songs
WITH (kafka_topic='psongs', key_format='AVRO', value_format='AVRO')
AS SELECT struct(artist := artist, album := album) as k,
           title, duration, type, release
FROM `raw_songs`
WHERE title IS NOT NULL AND album IS NOT NULL
PARTITION BY struct(artist := artist, album := album);
emit changes;

print 'psongs' from beginning limit 1;
```